6장. 로그인과 인증 구현하기

보안과 세션 관리는 웹 백엔드의 핵심 요소입니다. 본 장에서는 현대식 웹/앱 아키텍처의 웹 인증 방식인 **JWT(JSON Web Token)**를 채택하여 Stateless 인증 시스템을 구성하고, OAuth2 규격을 사용해 인증된 사용자 정보에 접근하는 코드를 구성합니다.

6.1 JWT(JSON Web Token) 방식의 이해

JWT는 헤더(Header), 페이로드(Payload), 서명(Signature)의 세 부분으로 구성되며, 점(`.`)으로 연결된 인코딩된 문자열입니다. 인증 시 서버가 클라이언트에 발급한 JWT를 전달받으면, 서버는 데이터베이스 조회 없이 자체 Secret Key로 서명의 조작 여부(유효성)만을 빠르게 검증하여 권한을 파악(Stateless)하므로 서버의 자원을 극도로 아낄 수 있습니다.

6.2 JWT 토큰 관리 모듈 개발

JWT 토큰을 생성하고 만료 시점을 주며 검증할 `jwt_handler.py` 모듈을 작성합니다. `python-jose` 혹은 `PyJWT` 라이브러리를 활용합니다.

```
# jwt_handler.py
import jwt
from datetime import datetime, timedelta, timezone

# ⚠️ 운영 배포 시에는 비밀키를 환경 변수로 감춰야 합니다.
SECRET_KEY = "SUPER_SECRET_FASTAPI_KEY_12345"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

def create_access_token(data: dict) -> str:
    """JWT 엑세스 토큰을 생성합니다."""
    to_encode = data.copy()
    # UTC 기준 만료 시점 설정
    expire = datetime.now(timezone.utc) + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})

    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt

def verify_token(token: str) -> dict | None:
    """JWT 토큰의 서명 및 만료 일자를 확인합니다."""
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        return payload
    except jwt.PyJWTError:
        return None  # 서명이 유효하지 않거나 만료된 토큰
```

6.3 로그인 및 토큰 발급 API 구현

이메일과 비밀번호를 검증하여 일치할 경우 JWT Access Token을 발급하는 로그인 API를 구현합니다.

```python
# routers/auth.py
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm
from sqlalchemy.orm import Session
from database import SessionLocal
from models import UserModel
from security import verify_password
from jwt_handler import create_access_token
from pydantic import BaseModel

router = APIRouter(tags=["auth"])

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

class Token(BaseModel):
    access_token: str
    token_type: str

# 🔑 OAuth2 표준 요청 폼(OAuth2PasswordRequestForm) 활용 로그인 엔드포인트
@router.post("/token", response_model=Token)
def login_for_access_token(
    form_data: OAuth2PasswordRequestForm = Depends(),
    db: Session = Depends(get_db)
) -> dict:
    # form_data.username 에 사용자가 입력한 이메일이 매핑되어 들어옴
    user = db.query(UserModel).filter(UserModel.email == form_data.username).first()
    if not user or not verify_password(form_data.password, user.hashed_password):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="이메일 또는 비밀번호가 잘못되었습니다.",
            headers={"WWW-Authenticate": "Bearer"},
        )

    # 토큰에 실을 Payload 데이터 설정
    access_token = create_access_token(data={"sub": user.email})
    return {"access_token": access_token, "token_type": "bearer"}
```

6.4 OAuth2PasswordBearer를 통한 인증 의존성 구축

FastAPI는 클라이언트 요청 헤더 중 `Authorization: Bearer ` 포맷에서 자동으로 토큰 값을 파싱하는 `OAuth2PasswordBearer` 스키마를 탑재하고 있습니다. 이를 사용해 인증된 사용자 객체를 구하는 의존성을 만듭니다.

```
from fastapi.security import OAuth2PasswordBearer
from jwt_handler import verify_token

# 로그인 엔드포인트 주소를 OAuth2PasswordBearer에 등록
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db)
) -> UserModel:
    """토큰을 파싱해 현재 사용자의 DB 객체를 반환합니다. 인증되지 않은 사용자는 401 차단."""
    payload = verify_token(token)
    if not payload:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="유효하지 않거나 만료된 토큰입니다.",
            headers={"WWW-Authenticate": "Bearer"},
        )
    email: str = payload.get("sub")
    user = db.query(UserModel).filter(UserModel.email == email).first()
    if not user:
        raise HTTPException(status_code=404, detail="사용자를 찾을 수 없습니다.")
    return user
```

```
# 사용법 예제: 로그인한 사용자만 할 일(Todo) 목록을 등록할 수 있도록 보호
from models import TodoModel

@router.post("/todos", status_code=201)
def create_todo_for_user(
    title: str,
    db: Session = Depends(get_db),
    current_user: UserModel = Depends(get_current_user) # 🔐 인증 보호 주입!
):
    print(f"로그인 사용자: {current_user.email}")
    new_todo = TodoModel(title=title, owner_id=current_user.id) # 소유권 연동 예시
    db.add(new_todo)
    db.commit()
    return {"status": "success"}
```